

Praktikum 1 - Matakuliah Pilihan Web (API Development)

Program Studi: Teknik Informatika

Nim: 3312401043

Nama: Salsa Putri Ajriyanti

Kelas: IF 4A Pagi web

Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukkan hasil akhir dari men-share repository github yang telah dibuat.

A. Setup Laravel dan Mysql

1. Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi [12.x](#) atau yang lebih baru)
2. Mempersiapkan database dan buat database baru bernama dblaravelapi
3. Update .env dan hubungkan dengan nama database dblaravelapi
4. Lakukan migrasi dengan menggunakan php artisan migrate
5. Pastikan laravel sudah terhubung dengan tabel dengan mengecek tabel yang dibuat laravel dalam mysql

B. Membuat Controller

1. Buatlah sebuah controller dengan nama Api/ProductController **php artisan make:controller Api/ProductController**
(Pastikan ada di dalam folder Api)
Pada bootstrap/app.php tambahkan setelah code web: __DIR__ api:
__DIR__.'../routes/api.php',

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'../routes/web.php',
        api: __DIR__.'../routes/api.php',
        commands: __DIR__.'../routes/console.php',
        health: '/up',
    );
```

2. Tambahkan Routes baru di router/api.php
Route::get('/products', [ProductController::class, 'index']);
(Apabila file api.php belum ada, clone web.php dan hapus routes "/") 3. Test API dengan mengunjungi localhost:8000/api/products

C. Gunakan database dan Model

1. Pastikan database sudah terkoneksi dengan laravel.
2. Buatlah sebuah tabel dengan nama `tblproducts` dengan field (`id`, `nama`, `deskripsi`, `foto`, dan `harga`) untuk tipe data menyesuaikan.
3. Tambahkan minimal 10 data untuk produk dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.
4. Buat model bernama `Product` **php artisan make:model Product**
5. Hubungkan model `Product` dengan `tblproduct` dengan mengisi pada class `Product`

```
class Product extends Model
{
    protected $table = 'tblproducts';
    protected $primaryKey = 'id';
    public $timestamps = false;

    // Field yang boleh diisi
    protected $fillable = [
        'nama',
        'deskripsi',
        'foto',
        'harga',
    ];
}
```

6. Pada `Product` controller ubah dalam fungsi `index`, dapatkan data dari model `Product` Dengan menambahkan `$products = Product::all();` Dan lakukan `return data => $products`

D. Membuat CRUD standar API Resource

1. Ubah `api.php` dengan api Resource yang sesuai standar laravel

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\ProductController;

// Route::get('/products', [ProductController::class, 'index']);

Route::apiResource('products', ProductController::class);
```

Dengan apiResource kamu tidak perlu menambahkan routes satu persatu untuk ProductController seperti create, update, delete, search dan view. Namun ada syarat penamaan fungsi agar routes ini bekerja yaitu **index, show, store, update, dan destroy**.

2. Ubah ProductController untuk fungsi-fungsi berikut **index, show, store, update, dan destroy**.

```
// ◆ GET /products
public function index()
{
    $products = Product::all();

    return response()->json([
        'status' => true,
        'data' => $products
    ]);
}
```

Gambar 04: Fungsi Index pada ProductsController

```
// ◆ GET /products/{id}
public function show($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    return response()->json([
        'status' => true,
        'data' => $product
    ]);
}
```

Gambar 05: Fungsi Show pada ProductsController

```
// ◆ POST /products
public function store(Request $request)
{
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);

    $product = Product::create($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil ditambahkan',
        'data' => $product
    ]);
}
```

Gambar 06: Fungsi Store pada ProductsController

```
// ◆ PUT /products/{id}
public function update(Request $request, $id)
{
    $product = Product::find($id);
    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);
    $product->update($request->all());
    return response()->json([
        'status' => true,
        'message' => 'Product berhasil diupdate',
        'data' => $product
    ]);
}
```

Gambar 07: Fungsi Update pada ProductsController

```
// ◆ DELETE /products/{id}
public function destroy($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    $product->delete();

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil dihapus'
    ]);
}
```

Gambar 08: Fungsi Destroy pada ProductsController

E. API Documentation

1. Install Scramble dengan menggunakan composer `composer require dedoc/scramble`
2. Masukan ke vendor laravel project anda `php artisan vendor:publish --provider="Dedoc\Scramble\ScrambleServiceProvider"`
3. Jalankan server (artisan serve)
4. Akses API documentation pada browser ke <http://localhost:8000/docs/api>
5. Untuk menambahkan deksripsi setiap API Endpoints bisa menggunakan DocBlock diatas fungsi dalam controller
6. Menambahkan DocBlock (Title dan Description)

```

class ProductController extends Controller
{
    /**
     * Ambil semua produk.
     *
     * Endpoint ini mengembalikan seluruh data produk yang tersedia.
     */

    // ◆ GET /products
    public function index()
    {
        $products = Product::all();

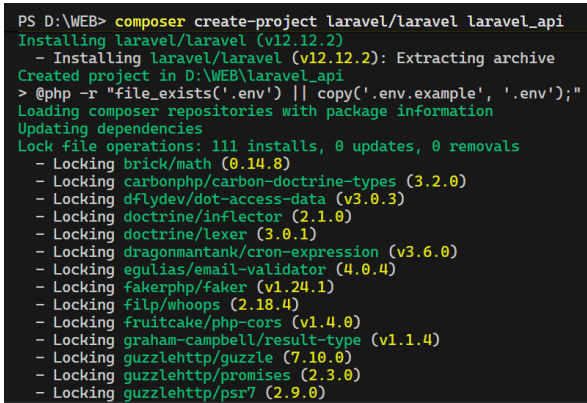
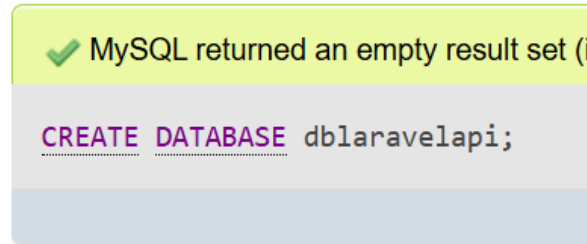
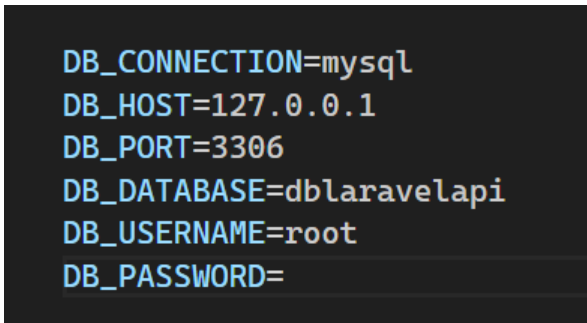
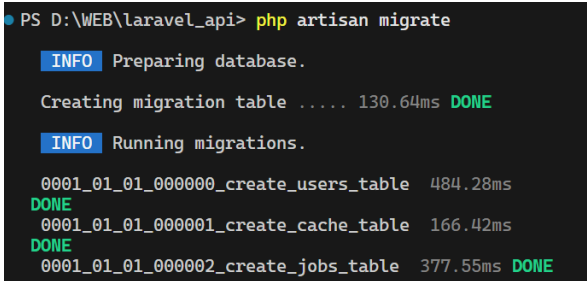
        return response()->json([
            'status' => true,
            'data' => $products
        ]);
    }
}

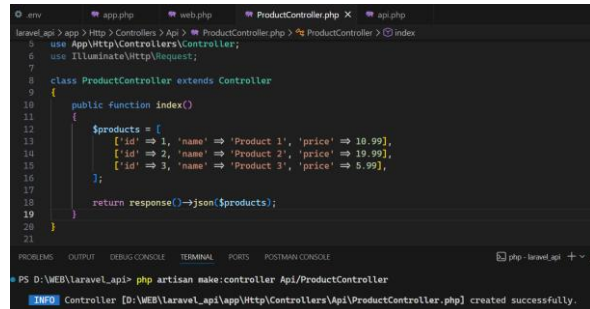
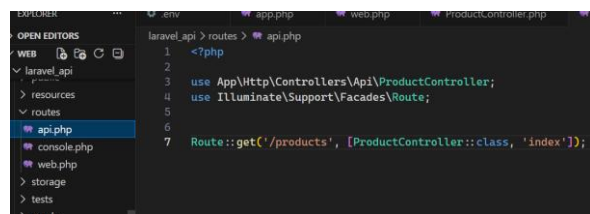
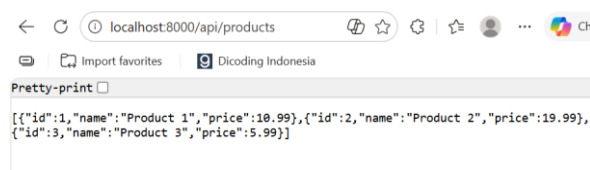
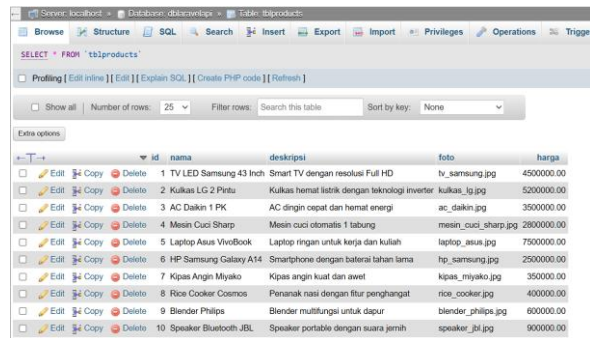
```

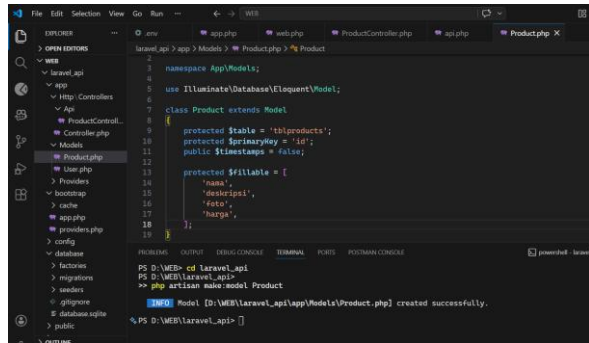
Gambar 08: DocBlock untuk title node index dan deksripsi

7. Refresh browser ke <http://localhost:8000/docs/api>
8. Tambahkan Title dan Deskripsi pada semua Endpoints

Hasil Pengerjaan

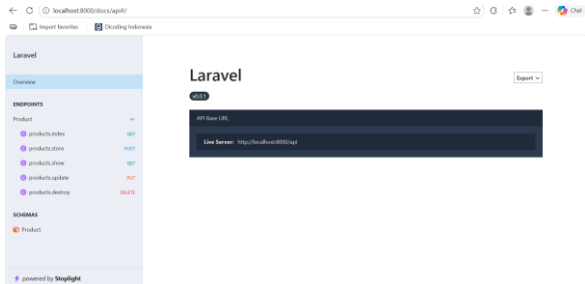
No.	Instruksi	Screenshot	Kendala/Saran
A.	Setup Laravel dan Mysql		
1.	Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi 12.x atau yang lebih baru)	 <pre>PS D:\WEB> composer create-project laravel/laravel laravel_api Installing laravel/laravel (v12.12.2) - Installing laravel/laravel (v12.12.2): Extracting archive Created project in D:\WEB\laravel_api > @php -r "file_exists('.env') copy('.env.example', '.env');" Loading composer repositories with package information Updating dependencies Lock file operations: 111 installs, 0 updates, 0 removals - Locking brick/math (0.14.8) - Locking carbonphp/carbon-doctrine-types (3.2.0) - Locking dflydev/dot-access-data (v3.0.3) - Locking doctrine/inflector (2.1.0) - Locking doctrine/lexer (3.0.1) - Locking dragonmantank/cron-expression (v3.6.0) - Locking egulias/email-validator (4.0.4) - Locking fakerphp/faker (v1.24.1) - Locking filp/whoops (2.18.4) - Locking fruitcake/php-cors (v1.4.0) - Locking graham-campbell/result-type (v1.1.4) - Locking guzzlehttp/guzzle (7.10.0) - Locking guzzlehttp/promises (2.3.0) - Locking guzzlehttp/psr7 (2.9.0)</pre>	
2.	Mempersiapkan database dan buat database baru bernama dblaravelapi	 <pre>✓ MySQL returned an empty result set (i CREATE DATABASE dblaravelapi;</pre>	
3.	Update .env dan hubungkan dengan nama database dblaravelapi	 <pre>DB_CONNECTION=mysql DB_HOST=127.0.0.1 DB_PORT=3306 DB_DATABASE=dblaravelapi DB_USERNAME=root DB_PASSWORD=</pre>	
4.	Lakukan migrasi dengan menggunakan php artisan migrate Pastikan laravel sudah terhubung dengan tabel dengan mengecek tabel yang dibuat laravel dalam mysql	 <pre>PS D:\WEB\laravel_api> php artisan migrate INFO Preparing database. Creating migration table 130.64ms DONE INFO Running migrations. 0001_01_01_000000_create_users_table 484.28ms DONE 0001_01_01_000001_create_cache_table 166.42ms DONE 0001_01_01_000002_create_jobs_table 377.55ms DONE</pre>	

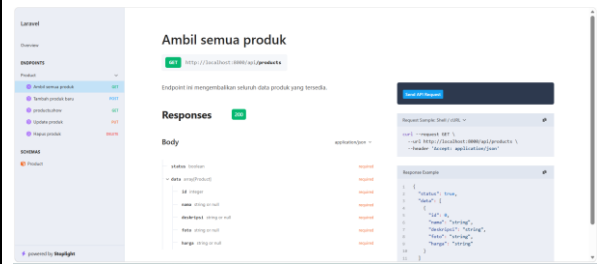
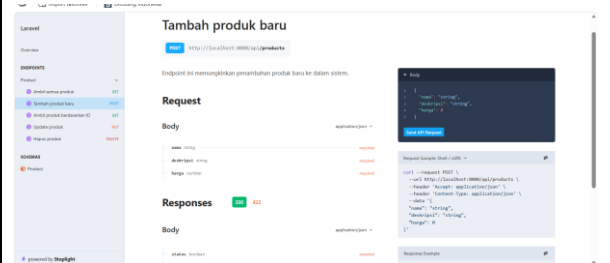
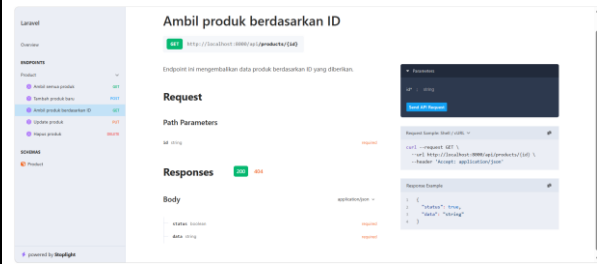
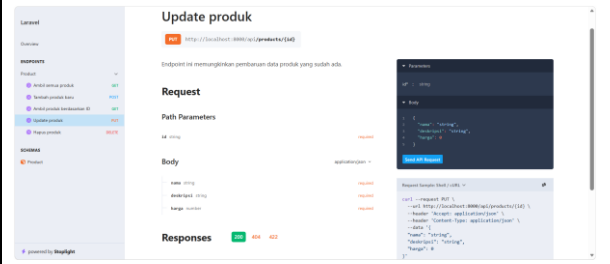
B	Membuat Controller		
1.	Buatlah sebuah controller dengan nama Api/ProductController php artisan make:controller Api/ProductController (Pastikan ada di dalam folder Api)		
2.	Pada bootstrap/app.php tambahkan setelah code web: __DIR__ api: __DIR__.'../routes/api.p hp',		
3.	Tambahkan Routes baru di router/api.php Route::get('/products', [ProductController::class, 'index']); (Apabila file api.php belum ada, clone web.php dan hapus routes "/)		
4.	Test API dengan mengunjungi localhost:8000/api/produ cts		
C	Gunakan database dan model		
1.	Database sudah terkoneksi dan Buatkan sebuah tabel dengan nama tblproducts dengan field (id, nama, deskripsi, foto, dan harga) untuk tipe data menyesuaikan. Tambahkan minimal 10 data untuk produk		

	dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.		
2.	Buat model bernama Product.php artisan make:model Product serta Hubungkan model Product dengan tblproduct dengan mengisi pada class Product		
3.	Pada Product controller ubah dalam fungsi index, dapatkan data dari model Product Dengan menambahkan \$products = Product::all(); Dan lakukan return data => \$products	<pre> <?php namespace App\Http\Controllers\Api; use App\Http\Controllers\Controller; use Illuminate\Http\Request; use App\Models\Product; class ProductController extends Controller { public function index() { \$products = Product::all(); return response()->json(\$products); } } </pre>	
D	Membuat CRUD standar API resource		
1.	Ubah api.php dengan api resource yang sesuai standar laravel	<pre> use Illuminate\Support\Facades\Route; use App\Http\Controllers\Api\ProductController; // Route::get('/products', [ProductController::class, 'index']); Route::apiResource('products', ProductController::class); </pre>	Dengan apiResource kamu tidak perlu menambahkan routes satu persatu untuk ProductController seperti create, update, delete, search dan view. Namun ada syarat penamaan fungsi agar routes ini bekerja yaitu index, show, store, update, dan destroy

2. Ubah ProductController untuk fungsi-fungsi berikut index, show, store, update, dan destroy.

```
laravel_api > app > Http > Controllers > Api > ProductController.php > ProductC
3  D:\WEB\laravel_api pp\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use Illuminate\Http\Request;
7  use App\Models\Product;
8
9  class ProductController extends Controller
10 {
11
12     //get products
13     public function index()
14     {
15         $products = Product::all();
16
17         return response()->json([
18             'status' => true,
19             'data' => $products
20         ]);
21     }
22
23     //get /products/{id}
24     public function show($id){
25         $product = Product::find($id);
26
27         if(!$product) {
28             return response()->json([
29                 'status' => false,
30                 'message' => 'Product tidak ditemukan'
31             ], 404);
32         }
33
34         return response()->json([
35             'status' => true,
36             'data' => $product
37         ]);
38     }
39
40     //post /products
41     public function store(Request $request){
42         $request->validate([
43             'nama' => 'required',
44             'deskripsi' => 'required',
45             'harga' => 'required|numeric'
46         ]);
47
48         $product = Product::create($request->all());
49
50         return response()->json([
51             'status' => true,
52             'message' => 'Product berhasil ditambahkan',
53             'data' => $product
54         ]);
55     }
56
57     //put /products/{id}
58     public function update(Request $request, $id){
59
60         $product = Product::find($id);
61         if (!$product) {
62             return response()->json([
63                 'status' => false,
64                 'message' => 'Product tidak ditemukan'
65             ], 404);
66         }
67         $request->validate([
68             'nama' => 'required',
69             'deskripsi' => 'required',
70             'harga' => 'required|numeric'
71         ]);
72
73         $product->update($request->all());
```

		<pre> 72 73 \$product->update(\$request->all()); 74 return response()->json([75 'status' => true, 76 'message' => 'Product berhasil diupdate', 77 'data' => \$product 78]); 79 } 80 81 //delete /products/{id} 82 public function destroy(\$id){ 83 84 \$product = Product::find(\$id); 85 86 if (!\$product) { 87 return response()->json([88 'status' => false, 89 'message' => 'Product tidak ditemukan' 90], 404); 91 } 92 93 \$product->delete(); 94 return response()->json([95 'status' => true, 96 'message' => 'Product berhasil dihapus' 97]); 98 } 99 100 } </pre>	
E	API Documentation		
1.	Install Scramble dengan menggunakan composer composer require dedoc/scramble	<pre> >> composer require dedoc/scramble ./composer.json has been updated Running composer update dedoc/scramble Loading composer repositories with package information Updating dependencies Lock file operations: 3 installs, 0 updates, 0 removals - Locking dedoc/scramble (v0.13.17) - Locking phpstan/phpdoc-parser (2.3.2) - Locking spatie/laravel-package-tools (1.93.0) Writing lock file Installing dependencies from lock file (including require-dev) Package operations: 3 installs, 0 updates, 0 removals - Downloading spatie/laravel-package-tools (1.93.0) - Downloading phpstan/phpdoc-parser (2.3.2) - Downloading dedoc/scramble (v0.13.17) - Installing spatie/laravel-package-tools (1.93.0): Extracting archive - Installing phpstan/phpdoc-parser (2.3.2): Extracting archive - Installing dedoc/scramble (v0.13.17): Extracting archive Generating optimized autoload files > Illuminate\Foundation\ComposerScripts::postAutoloadDump > @php artisan package:discover --ansi </pre>	
2.	Masukan ke vendor laravel project anda php artisan vendor:publish --provider="Dedoc\Scramble\ScrambleServiceProvider"	<pre> PS D:\WEB\laravel_api> php artisan vendor:publish --provider="Dedoc\Scramble\ScrambleServiceProvider" </pre>	
3.	Jalankan server	Php artisan serve	
4.	Akses API documentation pada browser ke http://localhost:8000/docs/api		

5.	Menambahkan DocBlock (title dan description) pada semua endpoints	<pre> /** * Hapus produk * * Endpoint ini memungkinkan penghapusan produk dari sistem. */ //delete /products/{id} public function destroy(\$id){ } /** * Update produk * * Endpoint ini memungkinkan pembaruan data produk yang sudah ada. */ //put /products/{id} public function update(Request \$request, \$id){ } /** * Tambah produk baru * * Endpoint ini memungkinkan penambahan produk baru ke dalam sistem. */ //post /products public function store(Request \$request){ </pre>	
6.	Overview		
	Get produk		
	Tambah produk		
	Ambil produk berdasarkan ID		
	Update produk		

Hapus produk

Lokal

Overview

Endpoints

Product

Product

Hapus produk

DELETE http://localhost:8080/api/products/145

Endpoint ini memungkinkan penghapusan produk dari sistem.

Request

Path Parameters

14 string

Responses

Body

Parameter

string

Request Example

curl -XDELETE http://localhost:8080/api/products/145 -H "Accept: application/json"

Response Example

{ "status": "success", "message": "Produk berhasil dihapus" }